Lab 12 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can verify at the terminal.

Question 1 — Three ReplicaSets, three Pods

Answer. Three ReplicaSets (one per version of the template) and **three** Pods, all in the most recent ReplicaSet. `kubectl get rs` shows the latest at `3 3 3` and the other two at `0 0 0`. The old ones are kept because they **are** the history: `rollout undo` simply sets the replicas of an old ReplicaSet back to 3 and those of the current one to 0. `revisionHistoryLimit` (10 by default) in the Deployment's spec bounds their number.

Why. A ReplicaSet at zero costs only one object in etcd; keeping it makes rollback instant, with no template to rebuild.

Nuance. Two updates that end up on the **same** template (1.0 → 1.1 → 1.0) reuse the existing ReplicaSet: the template hash is identical.

Example.

```
kubectl get rs -l app=api   # api-6c7bb54fc5 0 0 0 / api-5c8d86977d 0 0 0 / api-… 3 3 3
```

Question 2 — Selector and labels mismatch

Answer. `The Deployment "api" is invalid: spec.template.metadata.labels: Invalid value: … selector does not match template labels`. The rule: the Deployment's `selector` **must** select the Pods its template produces, otherwise it would create Pods it does not recognise as its own. If the cluster accepted it, the ReplicaSet would always see `0` Pods with `app: api`, create `app: backend` Pods endlessly without counting them — a Pod leak.

Why. The Deployment → Pods relationship is not ownership but **selection**; it must be consistent from creation.

Nuance. Conversely, a Pod created by hand with `app: api` would be **adopted** by the ReplicaSet (and a surplus Pod would be deleted): labels are a contract, not decoration.

Example.

```
kubectl apply -f deployment-wrong.yaml 2>&1 | grep -o 'selector does not match template labels'
```

Question 3 — The sequence of a rolling update

Answer. With `maxSurge: 25 %` and `maxUnavailable: 25 %` on three replicas (rounded: 1 extra, 0 fewer), the Events are: new RS `0 → 1`; once that Pod is ready, new `1 → 2` and old `3 → 2`; then old `2 → 1`, new `2 → 3`; finally old `1 → 0`. At no point fewer than three ready Pods. Creating three new Pods at once would require six Pods (exceeds `maxSurge`) or delete three old ones before the new are ready (exceeds `maxUnavailable`).

Why. The Deployment controller only adjusts the replicas of two ReplicaSets, step by step, within the two bounds and according to the Pods' `Ready` state.

Nuance. The readiness probe is what makes "ready" meaningful; without it, the rollout goes faster and may cut the service while the JVMs start.

Example.

```
kubectl get events --sort-by=.lastTimestamp | grep ScalingReplicaSet | tail -6
```

Question 4 — A single Pod in `Error`

Answer. The Deployment created **one** Pod of the new version (`maxSurge` = 1) and waits for it to be ready before removing an old one; since it exits in error in a loop, the rollout **stops there**, and the three old ones keep serving — that is the safety net. `kubectl rollout status` waits then times out: `error: timed out waiting for the condition`. Cause: `kubectl logs rs/api-6c87f96dd5 --tail=1` (or `kubectl logs api-6c87f96dd5-4v6s6 --previous`) → `demarrage impossible : configuration manquante`. Repair: `kubectl rollout undo deployment/api`.

Why. `maxUnavailable: 25 %` of 3 = 0: no old Pod may be removed until a new one is ready.

Nuance. Without a readiness probe, a container that **starts** and then crashes a few seconds later could have been "ready" long enough to remove an old one; the probe closes that window.

Example.

```
kubectl get deployment api      # READY 3/3   UP-TO-DATE 1   AVAILABLE 3
kubectl rollout undo deployment/api && kubectl rollout status deployment/api
```

Question 5 — Liveness on a dependency

Answer. When the database goes down, `/actuator/health` turns `DOWN`, the liveness fails three times, the kubelet **kills and restarts** the API; the API restarts, still cannot find the database, gets killed again: `RESTARTS` climbs, then `CrashLoopBackOff`. For five minutes, the API, which could have served requests without the database (static pages, cache, clean errors), is unreachable, and when the database returns it takes several minutes to get out of the backoff. There should have been a liveness on `/actuator/health/liveness` (internal state of the process, independent of dependencies) and, if needed, a **readiness** on the database: the Pod leaves the traffic without being killed.

Why. Restarting only fixes failures **inside** the process; a missing dependency is not fixed by a restart.

Nuance. Spring Boot separates the two precisely for this: `LivenessState` depends on no external `HealthIndicator`.

Example.

```
kubectl describe pods -l app=api | grep -E 'Liveness probe failed|Killing' | tail -2
```

Question 6 — Broken readiness, broken liveness

Answer. Readiness on `8081`: one new Pod `0/1 Running` (never ready), the three old ones `1/1`, `kubectl get deployment` shows `READY 3/3 UP-TO-DATE 1 AVAILABLE 3`; the rollout is stuck, the service **answers** through the old Pods. Liveness on `8081`: the new Pods become `Ready` (the

readiness does succeed), the rollout **proceeds** and replaces the three old ones, then the liveness kills each new Pod: `0/1 CrashLoopBackOff` × 3, service **dead**.

Why. The rollout only waits for readiness; liveness acts afterwards, on Pods already counted as good.

Nuance. That is the argument for testing a probe change in staging, and for giving the liveness a generous `initialDelaySeconds` and `failureThreshold`.

Example.

```
kubectl get pods            # readiness: api-xxx 0/1 Running + 3 old 1/1 ; liveness: 3 × CrashLoopBackOff
kubectl rollout undo deployment/api
```

Question 7 — `CrashLoopBackOff`

Answer. A state of the **kubelet**: the container exits with an error, the kubelet relaunches it, it exits again, and the kubelet **doubles the delay** before each relaunch (10 s, 20 s, 40 s… capped at five minutes) so as not to saturate the node. The application has only one state: it crashes. The cause is in `kubectl logs <pod> --previous`, the output of the previous attempt; `kubectl logs` without options shows the **current** container, often empty if it has just been relaunched or has not written anything yet.

Why. The backoff is a protection for the node, not a diagnosis: nothing in the delay says why the process exits.

Nuance. The `RESTARTS` counter resets to zero when the container runs ten minutes without exiting: an old restart may therefore no longer show.

Example.

```
kubectl get pod api-xxx -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
kubectl logs api-xxx --previous | tail -3
```

Question 8 — `HEALTHCHECK` versus readiness

Answer. The `HEALTHCHECK` is run **inside** the container by the engine (Podman through a timer, Docker through the daemon): the tool (`wget`, `curl`) must be in the image; a failure changes the `(unhealthy)` state and nothing else, except with Compose `depends_on`. The `httpGet` `readinessProbe` is run by the **kubelet**, from the node, with an HTTP request to the Pod's IP: no tool in the image; a failure removes the Pod from traffic. A distroless image (no shell, no `wget`) cannot carry a working `HEALTHCHECK`; it works perfectly with an `httpGet` or `tcpSocket` probe.

Why. Kubernetes moved the check from the container to the platform; that is why it ignores the images' `HEALTHCHECK`.

Nuance. The `exec` probe takes over the Docker model (command inside the container) for cases with no HTTP port.

Example.

```
kubectl explain pod.spec.containers.readinessProbe.httpGet | head -8
```

Question 9 — Revisions with gaps

Answer. Every change of the template creates a revision, and every `rollout undo` does too: going back to revision 1 **creates** a revision 3 identical to 1 (the ReplicaSet of 1 is reused and renumbered). The missing numbers are those the rollbacks "consumed": `1 → 3`, `2 → 4`, and so on. It is not a problem: the history stays consistent, and `rollout undo --to-revision=N` works with the numbers displayed.

Why. A revision is a `deployment.kubernetes.io/revision` annotation on a ReplicaSet; a reactivated ReplicaSet gets the next number.

Nuance. The empty `CHANGE-CAUSE` column is filled with `kubectl annotate deployment api kubernetes.io/change-cause="move to 1.1"`, precious in an incident review.

Example.

```
kubectl get rs -o custom-
columns=NAME:.metadata.name,REV:.metadata.annotations.deployment\.kubernetes\.io/revision,REPLI
CAS:.spec.replicas
```

Question 10 — `maxSurge` and `maxUnavailable`

Answer. `maxSurge: 0, maxUnavailable: 1`: the cluster first removes an old Pod, then creates a new one; peak 3, trough **2** Pods — never more than three, hence suited to full nodes, at the price of reduced capacity during the update. `maxSurge: 1, maxUnavailable: 0`: it creates a new Pod, waits for it to be ready, then removes an old one; peak **4**, trough 3 — never fewer than three ready, hence suited to a service that must not lose capacity, at the price of one more Pod to house.

Why. The two bounds read like a budget: how much I may exceed, how much I may lack.

Nuance. `maxSurge: 0, maxUnavailable: 0` is rejected: the rollout could never begin.

Example.

```
kubectl patch deployment api -p '{"spec":{"strategy":{"rollingUpdate":
{"maxSurge":1,"maxUnavailable":0}}}}'
```

Question 11 — Two `Pending` Pods

Answer. The node has no more **resources** for them (memory or CPU, if the template has `requests`, lab 16) or has reached its Pod limit; `kubectl describe pod` shows `FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`. A rollout in that state proceeds **partially**: the Pods that can be scheduled are replaced, the `Pending` ones stay so (for the new version), and `rollout status` never completes.

Why. The scheduler only places a Pod if a node can satisfy its requests; `scale` creates no capacity.

Nuance. Without `requests`, a node accepts Pods up to its count limit (110 by default): the `Pending` ones then come from something else (volume, affinity, taint).

Example.

```
kubectl describe node | grep -A6 'Allocated resources'
```

Question 12 — Editing a Pod by hand

Answer. At the next **rollout**, the Pod is replaced by a fresh Pod built from the template: the change disappears. At the next **restart** of the container (liveness, crash), the writable layer's filesystem is recreated from the image: gone too. The **other two replicas** never saw the change: the service behaves differently depending on which Pod answers. The configuration belonged in a **ConfigMap** or a **Secret** (lab 14) referenced by the template, letting the rollout propagate the change to every replica.

Why. A Pod is disposable (lab 06); only the spec is durable.

Nuance. `kubectl exec` remains legitimate for **observing** and for a temporary diagnosis, never for configuring.

Example.

```
kubectl rollout restart deployment/api    # recreates every Pod from the template: the tinkering disappears
```

Question 13 — A bare Pod in production

Answer. (1) **No self-healing**: deleted, or its node lost, the Pod does not come back. (2) **No zero-downtime update**: changing the image requires `delete` then `apply`. (3) **No rollback**: no history. (4) **No scaling**: moving to two instances requires a second manifest. The only legitimate case: a **one-off job** or a test, which a `Job` does better anyway; for a service, even with one instance, the Deployment costs only five more lines.

Why. The Pod is a unit of execution, not a management object; controllers add the management.

Nuance. A bare Pod **is** recreated by the kubelet if it is a *static Pod* (a file on the node) — that is how the control plane components run in some clusters, not your applications.

Example.

```
kubectl create deployment api --image=localhost/api:1.0 --replicas=1 --dry-run=client -o yaml | head -20
```